
MiniGames App

TP note – Seance 3 sur 3

Developpement Android avec Jetpack Compose

Theme de la seance :

Persistence des donnees

Room, scores, leaderboard

Formation : CNAM – 1ere annee

Travail : En binome

Duree : 4h

Auteur : Jordan Hiertz
hiertzjordan@gmail.com

Ce sujet est le troisieme et dernier volet d'un TP progressif sur 3 seances.
Vous repartez du code rendu lors de la seance 2.

Vue d'ensemble

Vous allez finaliser votre **MiniGames App** en y integrant la persistance locale des donnees avec **Room**. Les scores de chaque partie seront sauvegardes en base de donnees et accessibles depuis un ecran **leaderboard**.

Progression sur 3 seances

Seance 1	Kotlin + Compose	Ecran d'accueil, jeu de reaction
Seance 2	Architecture + Navigation	ViewModel, NavHost, Mot cache
Seance 3	Base de donnees	Room, scores, leaderboard

A la fin de cette seance, l'application permettra a chaque joueur de saisir son pseudo, de jouer aux deux jeux existants, et de consulter un leaderboard affichant les dix meilleurs scores toutes parties confondues.

Objectifs pedagogiques

1. Comprendre le role de **Room** comme couche de persistance locale.
2. Creer une entite, un DAO et une classe de base de donnees.
3. Integrer Room dans l'architecture MVVM via un **Repository**.
4. Utiliser **viewModelScope** pour executer des requetes en arriere-plan.
5. Implementer un ecran leaderboard avec un nouveau **Screen** et **ViewModel**.

Modalites de rendu

Date limite : vendredi 12 juin 2025, 23h59.

Les projets seront clones le samedi matin, aucun commit apres cette date ne sera pris en compte.

Depot du code

Deposez votre projet sur un depot Git accessible (GitHub, GitLab...) et **envoyez le lien par Discord ou par mail** avant la date limite. Le depot peut etre prive ; dans ce cas, pensez a m'y donner acces.

Branche a livrer

Precisez dans le README la **branche contenant la version finale** a evaluer (**main**, **develop** ou autre). C'est cette branche qui sera clonee et testee.

README

Le README doit contenir a minima :

1. **Nom(s)** du binome ou de la personne.
2. **Branche** a utiliser pour tester le projet.
3. **Ce qui fonctionne** : listez les fonctionnalites implementees et operationnelles.
4. **Ce qui ne fonctionne pas** : et pourquoi.

Criteres de notation

Bareme – /20

Fonctionnel 12 pts

Jeu de reaction complet (TP1)	2 pts
Navigation multi-ecrans (TP2)	1 pt
Jeu de mots complet (TP2)	4 pts
Saisie du pseudo avec validation (TP3)	1 pt
Sauvegarde des scores via Room (TP3)	2 pts
Ecran leaderboard (TP3)	2 pts

Technique 6 pts

Architecture MVVM	2 pts
Qualite du code	3 pts
Git + README	1 pt

Bonus (plafonnes a 2 pts) +2 pts

TP1 – Timer aveugle
TP1 – Vitesse variable
TP2 – Indice (premiere lettre du mot)
TP3 – Filtre du leaderboard par jeu
TP3 – Statistiques personnelles
TP3 – Reinitialisation des scores

Qualite du code – exemples de criteres observes : nommage explicite des variables et fonctions, absence de code mort ou commente, separation claire entre logique metier et affichage, lisibilite generale.

Git + README – un historique de commits logiques est attendu. Un seul commit “projet final” en fin de seance ne sera pas valorise.

1. Mise en route

1.1. Nouvelles dependances

Dans `gradle/libs.versions.toml`, ajoutez Room et le plugin KSP :

```
[versions]
# ...
room = "2.7.1"
ksp = "2.0.21-1.0.28"

[libraries]
# ...
room-runtime = { group = "androidx.room", name = "room-runtime", version.ref = "room" }
room-compiler = { group = "androidx.room", name = "room-compiler", version.ref = "room" }
room-ktx = { group = "androidx.room", name = "room-ktx", version.ref = "room" }

[plugins]
# ...
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

Dans `build.gradle.kts` (module), ajoutez le plugin en haut du fichier et les dependances dans le bloc `dependencies` :

```
plugins {  
    // ...  
    alias(libs.plugins.ksp)  
}  
  
dependencies {  
    // ...  
    implementation(libs.room.runtime)  
    implementation(libs.room.ktx)  
    ksp(libs.room.compiler)  
}
```

Synchronisez le projet apres cette modification (*Sync Now*).

1.2. Structure des packages

Ajoutez les nouveaux fichiers a la structure existante :

```
com.votregroupe.minigames/  
|-- data/  
|   |-- Score.kt                (nouveau)  
|   |-- ScoreDao.kt            (nouveau)  
|   |-- AppDatabase.kt         (nouveau)  
|   '-- ScoreRepository.kt     (nouveau)  
|-- ui/  
|   |-- home/  
|   |   '-- HomeScreen.kt      (modifier)  
|   |-- reaction/  
|   |   |-- ReactionScreen.kt  (modifier)  
|   |   '-- ReactionViewModel.kt (modifier)  
|   |-- wordgame/  
|   |   |-- WordGameScreen.kt  (modifier)  
|   |   '-- WordGameViewModel.kt (modifier)  
|   '-- leaderboard/  
|       |-- LeaderboardScreen.kt (nouveau)  
|       '-- LeaderboardViewModel.kt (nouveau)  
|-- Navigation.kt              (modifier)  
'-- MainActivity.kt            (modifier)
```

2. Persistance avec Room

Room est une bibliotheque d'abstraction sur SQLite qui repose sur trois composants :

- **Entite** : une `data class` annotee `@Entity` qui represente une table. Chaque propriete devient une colonne.
- **DAO** : une interface annotee `@Dao` qui definit les operations sur la base. Room genere l'implementation a la compilation.
- **Database** : une classe abstraite annotee `@Database` qui sert de point d'entree et expose les DAO.

```
@Entity(tableName = "movie")
data class Movie(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    @ColumnInfo(name = "title") val title: String,
    @ColumnInfo(name = "director") val director: String,
    val rating: Float,
    val year: Int
)

@Dao
interface MovieDao {
    @Insert
    suspend fun insertMovie(movie: Movie)

    @Query("SELECT * FROM movie ORDER BY rating DESC LIMIT 10")
    suspend fun getTopRatedMovies(): List<Movie>
}

@Database(entities = [Movie::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun movieDao(): MovieDao
}
```

Room interdit l'accès à la base de données sur le thread principal pour ne pas bloquer l'interface utilisateur. Les méthodes du DAO sont marquées **suspend** : elles doivent être appelées depuis une coroutine.

`viewModelScope.launch` lance une coroutine liée au cycle de vie du ViewModel. Elle est automatiquement annulée quand le ViewModel est détruit.

```
class MonViewModel(application: Application) : AndroidViewModel(
    application) {

    private val repository = ScoreRepository(
        AppDatabase.getDatabase(application).scoreDao()
    )

    fun sauvegarderScore(nom: String, points: Int) {
        viewModelScope.launch { // lance la coroutine
            repository.insertScore( // suspend : s'exécute hors du
                thread UI
                Score(playerName = nom, gameName = "Reaction", score =
                    points)
            )
        }
    }
}
```

Remarque : vos ViewModels doivent désormais étendre `AndroidViewModel` (et non plus `ViewModel`) pour recevoir le contexte `application` nécessaire à l'initialisation de la base de données.

2.1. Entite Score

Créez `Score.kt` dans le package `data/`. L'entité doit contenir les champs suivants :

- `id` : cle primaire Long, générée automatiquement.
- `playerName` : pseudo du joueur (colonne `player_name`).
- `gameName` : nom du jeu (colonne `game_name`).
- `score` : score obtenu.
- `date` : timestamp de la partie, initialise automatiquement.

Annotez la classe et ses propriétés en vous appuyant sur l'encadre ci-dessus.

2.2. DAO

Créez `ScoreDao.kt`. Le DAO doit exposer deux méthodes :

```
@Dao
interface ScoreDao {

    suspend fun insertScore(score: Score) // Insere un score

    suspend fun getTopScores(): List<Score> // Top 10 par ordre decroissant
}
```

Ajoutez les annotations Room appropriées sur chaque méthode.

2.3. Base de donnees

Creez `AppDatabase.kt`. La classe doit :

- Etre annotee `@Database` en declarant `Score::class`.
- Etre abstraite et etendre `RoomDatabase`.
- Exposer `scoreDao()` comme methode abstraite.
- Implementer le pattern Singleton (revu dans le cours 07) pour garantir une instance unique. Nommez la base `"minigames_database"`.

2.4. Repository

Creez `ScoreRepository.kt`. Le Repository recoit un `ScoreDao` en parametre et expose les memes operations vers les ViewModels, en respectant la separation des responsabilites vue en cours.

3. Saisie du pseudo et navigation

3.1. Pseudo sur l'ecran d'accueil

Modifiez `HomeScreen` pour ajouter :

- Un `TextField` permettant au joueur de saisir son pseudo.
- Les boutons de jeu **desactives** tant que le pseudo est vide.
- Un bouton **Leaderboard** toujours actif.

3.2. Passage du pseudo via la navigation

Le pseudo doit etre transmis aux ecrans de jeu via les routes de navigation. Mettez a jour `Navigation.kt` pour ajouter la route `Leaderboard` et passer `playerName` en parametre des routes `Reaction` et `WordGame`, sur le meme modele que la seance 2.

Mettez ensuite a jour le `NavHost` et les signatures des composables `ReactionScreen` et `WordGameScreen` en consequence.

4. Sauvegarde des scores

4.1. Mise a jour des ViewModels

Note architecturale : dans ce TP, chaque ViewModel instancie son propre `ScoreRepository`. Cela fonctionne car `AppDatabase` est un singleton, mais en production on centraliserait le repository dans une classe ou via un framework d'injection de dependances (Hilt, Koin).

Modifiez `ReactionViewModel` et `WordGameViewModel` pour :

1. Etendre `AndroidViewModel` au lieu de `ViewModel`.
2. Initialiser le `ScoreRepository`.
3. Ajouter une methode `saveScore(playerName: String)` qui insere le score en base via `viewModelScope.launch`.

```

class ReactionViewModel(application: Application) : AndroidViewModel(
    application) {

    private val repository = // Initialisez le repository ici

    // ... etats existants ...

    fun saveScore(playerName: String) {
        viewModelScope.launch {
            repository.insertScore(
                Score(
                    playerName = playerName,
                    gameId     = "Reaction",
                    score      = /* votre score actuel */
                )
            )
        }
    }
}

```

4.2. Declenchement de la sauvegarde

Plutot que de declencher la sauvegarde depuis le composable, le ViewModel doit la gerer lui-meme en interne. Pour cela, passez `playerName` en parametre de `startGame()` et stockez-le dans le ViewModel. La sauvegarde est alors appelee automatiquement lors du passage en `GAME_OVER` :

```

class ReactionViewModel(application: Application) : AndroidViewModel(
    application) {

    private var playerName: String = ""

    fun startGame(playerName: String) {
        this.playerName = playerName
        // ... logique existante ...
    }

    // Appelee en interne lors du passage en GAME_OVER
    private fun saveScore() {
        viewModelScope.launch {
            repository.insertScore(
                Score(playerName = playerName, gameId = "Reaction", score =
                    /* score actuel */)
            )
        }
    }
}

```

Dans le composable, transmettez simplement le pseudo au demarrage via `LaunchedEffect(Unit)`, comme vous le faisiez deja en seance 2 :

```

LaunchedEffect(Unit) { viewModel.startGame(playerName) }

```

Appliquez le meme principe dans `WordGameViewModel` et `WordGameScreen`.

5. Ecran Leaderboard

5.1. LeaderboardViewModel

Creez `LeaderboardViewModel` en vous appuyant sur l'encadre `Coroutines`. Le `ViewModel` doit exposer la liste des scores via un `StateFlow` et charger les données automatiquement dans le bloc `init` :

```
class LeaderboardViewModel(application: Application) : AndroidViewModel(application) {  
    private val repository = // Initialisez le repository ici  
  
    private val _scores = MutableStateFlow<List<Score>>(emptyList())  
    val scores: StateFlow<List<Score>> = _scores.asStateFlow()  
  
    init {  
        // Chargez les scores ici via viewModelScope.launch  
    }  
}
```

5.2. LeaderboardScreen

Creez `LeaderboardScreen`. Le composable observe le `StateFlow` du `ViewModel` et affiche la liste dans une `LazyColumn`.

Chaque ligne doit afficher au minimum le **rang**, le **pseudo**, le **nom du jeu** et le **score**.

6. Ordre d'implémentation suggéré

Cet ordre est une suggestion – vous pouvez l'adapter selon votre avancement.

1. **Dependances et synchronisation** : ajoutez `Room` et `KSP`, vérifiez que le projet compile.
2. **Couche data** : créez `Score`, `ScoreDao`, `AppDatabase` et `ScoreRepository`. `Room` signale les erreurs à la compilation – vérifiez que le build passe avant de continuer.
3. **Pseudo sur HomeScreen** : ajoutez le `TextField`, désactivez les boutons si le pseudo est vide, et mettez à jour les routes de navigation pour transporter le pseudo.
4. **ReactionViewModel** : faites-le étendre `AndroidViewModel`, ajoutez le repository et `saveScore()`. Branchez la sauvegarde dans `ReactionScreen` via `LaunchedEffect`.
5. **WordGameViewModel** : même démarche que pour `Reaction`.
6. **LeaderboardViewModel + LeaderboardScreen** : implémentez le `ViewModel`, construisez l'écran et ajoutez le bouton d'accès depuis `HomeScreen`.
7. **Vérification end-to-end** : jouez une partie, vérifiez que le score apparaît dans le leaderboard.
8. **Polish** : formatez la date, soignez la mise en page du leaderboard.

Bonus

- **Filtre par jeu** : ajoutez des boutons pour filtrer le leaderboard par jeu ("Reaction" / "Mot cache" / "Tous"). Cela implique d'ajouter une requête adaptée dans le DAO.
- **Statistiques personnelles** : affichez pour le joueur courant son nombre de parties et son score moyen. Utilisez les fonctions d'agregation SQL `COUNT` et `AVG` dans de nouvelles méthodes du DAO.
- **Reinitialiser les scores** : ajoutez un bouton qui supprime tous les scores de la base de données.